Node.js Backend Development

Lecture 7 — Detailed English Notes

Restaurant application: MongoDB connection with Mongoose, schemas and models, parsing request bodies, and saving records with async/await.

Prepared from the provided lecture transcript. Code examples are organized and clarified for revision; where the transcript uses illustrative names, those names are retained where practical.

1. What this lecture builds

The previous lecture introduced MongoDB, NoSQL concepts, MongoDB Compass, and basic database operations. This lecture starts connecting those ideas to an Express/Node.js restaurant application. The central flow is:

```
Client (Postman / frontend)
        ↓ HTTP request
Express server and route handler
        ↓ Mongoose model
MongoDB database server
        ↓ result / error
Express sends an HTTP response to the client
```

The lecture first creates a dedicated database-connection file, then defines data structures with Mongoose schemas and models, and finally adds an endpoint that accepts JSON and saves a document.

2. Why keep database connection code in db.js?

A dedicated file such as **db.js** keeps connection logic separate from route logic. This is an example of organizing code by responsibility: the database file manages the connection, while the server file defines and handles HTTP endpoints.

• **Separation of concerns:** a developer can find and maintain connection logic in one place.

• **Reuse:** the server imports the connection module instead of repeating connection setup.

• **Readability:** the application becomes easier for teammates to understand as it grows.

The transcript uses Mongoose as the bridge between the Node.js application and MongoDB. Mongoose provides a higher-level interface for connecting, defining schemas/models, validating data, and performing database operations.

3. Connecting Node.js to MongoDB with Mongoose

3.1 Install and import Mongoose

Mongoose must be installed as a project dependency. The lecture notes that installed dependencies are listed in package.json. In a typical Node.js project, install it from the project directory:

```
npm install mongoose
```

In a CommonJS project, import it like this:

```
const mongoose = require('mongoose');
```

If a project is configured for ES modules, the equivalent import syntax is different. Use the module style already configured in your project rather than mixing styles.

3.2 Connection URI and database name

A MongoDB connection URI tells Mongoose where the database server is and which database to use. The transcript demonstrates a local MongoDB server and a database named **hotel** for the restaurant example. A common local URI format is:

```
mongodb://127.0.0.1:27017/hotel
```

The hostname identifies the local machine, 27017 is the usual MongoDB port, and **hotel** is the database name. If the database does not yet exist, MongoDB generally creates it when data is first written; simply specifying the name does not necessarily make it appear immediately in every database viewer.

The transcript mentions older connection options such as *useNewUrlParser* and *useUnifiedTopology*. These were used with older Mongoose/MongoDB driver versions. In modern versions, they are generally no longer needed. Follow the documentation for the version installed in your project.

3.3 Connect and keep the connection object

```
mongoose.connect(mongoURL);
const db = mongoose.connection;
```

Mongoose maintains a default connection object. The lecture calls it **db**. That object can be used to listen for connection events and inspect the connection state.

3.4 Connection events

Event listeners allow the application to react when the database connection changes. The lecture demonstrates logging a message when the database connects, disconnects, or reports an error.

```
db.on('connected', () => {
  console.log('Connected to MongoDB server');
});

db.on('error', (err) => {
  console.error('MongoDB connection error:', err);
});

db.on('disconnected', () => {
  console.log('MongoDB disconnected');
});
```

The transcript uses event labels conceptually to show that the application can detect connection-state changes. Mongoose event names should be spelled exactly as supported by the installed version; commonly used events include **connected**, **error**, and **disconnected**.

If MongoDB is stopped while the Node server is running, the application may log a disconnection. If MongoDB becomes available again, Mongoose may reconnect depending on the driver/version and configuration. A log message alone is not proof that a database write succeeded; it only reports the event being observed.

3.5 Export the connection module

```
// db.js
const mongoose = require('mongoose');
const mongoURL = 'mongodb://127.0.0.1:27017/hotel';

mongoose.connect(mongoURL);
const db = mongoose.connection;

db.on('connected', () => console.log('Connected to MongoDB'));
db.on('error', (err) => console.error('MongoDB error:', err));
db.on('disconnected', () => console.log('MongoDB disconnected'));

module.exports = db;
```

The server imports this file near startup. Requiring/importing the module executes its connection setup, so the application begins establishing the database connection before its routes need to save or retrieve records. In a production application, it is often better to await a successful connection before accepting traffic.

4. Schemas and models

4.1 What is a schema?

A **schema** is the blueprint for the shape of documents that the application expects in a collection. It defines field names, field types, and rules such as whether a field is required, whether it must be unique, or which values are allowed.

The lecture's example describes a person associated with the restaurant. Fields discussed include name, age, role, mobile number, email, address, and salary. The point is to decide what the record should contain before accepting data.

4.2 What is a model?

A **model** is created from a schema. It provides methods for creating, reading, updating, and deleting documents in the corresponding MongoDB collection. In the lecture, the Person model is exported so that server routes can use it.

```
const mongoose = require('mongoose');

const personSchema = new mongoose.Schema({
  name: { type: String, required: true },
```

```
  age: { type: Number },
  role: {
    type: String,
    enum: ['chef', 'waiter', 'manager'],
    required: true
  },
  mobile: { type: String, required: true },
  email: { type: String, unique: true },
  address: { type: String },
  salary: { type: Number }
});

const Person = mongoose.model('Person', personSchema);
module.exports = Person;
```

The code is an organized illustration of the schema fields discussed in the transcript. A mobile number is usually stored as a string because it may include a leading zero, country code, or formatting characters; it is not normally used for arithmetic.

4.3 Common schema rules

• **type:** specifies the expected type, such as String, Number, Boolean, Date, or an array/object.

• **required: true:** the field must be supplied for a valid document.

• **enum:** restricts a field to a list of permitted values. For example, role may be chef, waiter, or manager.

• **unique: true:** requests a unique index for the field. It is not an ordinary Mongoose validator; duplicate values can cause a MongoDB duplicate-key error, and the index must exist for uniqueness to be enforced.

Schema validation helps reject malformed data before it is saved. It does not replace all security checks or business rules, and a schema should match the actual needs of the application.

5. Create the model and use it in a route

The model is exported from its own file, for example **models/Person.js**, and imported in the server file:

```
const Person = require('./models/Person');
```

This avoids rewriting field definitions and gives route handlers a consistent interface to the collection. A typical project structure might look like:

```
project/
  db.js
  server.js
  models/
    Person.js
    MenuItem.js
  package.json
```

6. Middleware and request bodies

6.1 What is middleware?

Middleware is a function that runs during the request-response cycle, before a request reaches the final route handler or while it is being processed. The lecture introduces body-parser to explain how incoming request data is made available to the application.

6.2 Why parse the body?

When Postman or a frontend sends JSON in a POST request, the raw request body needs to be parsed. With Express, the usual modern setup is:

```
app.use(express.json());
```

After this middleware is registered, JSON data is available as **req.body** in matching route handlers. The middleware should be registered before the routes that need it.

The transcript refers to the body-parser package and the *bodyParser.json()* style. In current Express applications, **express.json()** is commonly used for JSON bodies without installing body-parser separately. Other body formats may require different middleware.

6.3 The request flow

```
Postman sends JSON
   ↓
express.json() parses the body
   ↓
req.body contains a JavaScript object
   ↓
Route creates a Mongoose model instance
   ↓
Document is saved to MongoDB
```

Without the appropriate body-parsing middleware, **req.body** may be undefined, so the route cannot read the submitted fields as expected.

7. POST endpoint: saving a person

The lecture creates a POST route, conceptually **POST /person**, which accepts a person's fields from the request body, creates a model instance, and saves it.

```
app.post('/person', async (req, res) => {
  try {
    const data = req.body;
    const newPerson = new Person(data);
    const savedPerson = await newPerson.save();

    console.log('Person data saved successfully');
    res.status(200).json(savedPerson);
  } catch (err) {
    console.error('Error saving person:', err);
    res.status(500).json({ error: 'Internal server error' });
  }
});
```

This is a cleaned-up illustrative version of the flow taught in the transcript. In a real API, successful creation is commonly reported with status **201 Created**. Invalid input can be returned as a 400-series response, while unexpected server/database failures may use 500.

7.1 Understanding each line

• **app.post('/person', ...)** registers a handler for POST requests to the /person route.

• **req.body** contains the parsed data sent by the client.

• **new Person(data)** creates a Mongoose document instance using the model's schema.

• **await newPerson.save()** waits for MongoDB to complete the save operation.

• **res.status(...).json(...)** sends an HTTP status code and JSON response back to the client.

• **try/catch** handles errors from asynchronous database operations so the server can return a meaningful response rather than leaving the request unresolved.

8. Why async/await is used

Saving or querying a database is asynchronous: it may take time, and the program should not pretend the result is available immediately. The lecture compares callback-based code with async/await and recommends async/await because it is often easier to read.

A function declared with **async** can use **await**. The await expression pauses that async function until the promise settles. If the operation succeeds, execution continues with its result; if it rejects, control moves to the catch block when it is inside a try/catch.

```
try {
  const savedPerson = await newPerson.save();
  res.status(201).json(savedPerson);
} catch (err) {
  res.status(500).json({ error: 'Internal server error' });
}
```

This pattern is useful for database operations, network requests, and file-system tasks. It does not block the entire Node.js process; it pauses the current async function while the promise is pending.

9. Callback approach versus async/await

The transcript first shows a callback-style save flow, then explains that the callback form is no longer the preferred approach in the version being demonstrated. Callback nesting can make error handling and response logic harder to follow. The async/await approach keeps the success path in one readable block and handles rejected operations in catch.

• Callback style: the next steps are placed inside a callback that receives an error/result.

• Async/await style: await the promise inside try, and handle failure in catch.

The broad lesson is not that callbacks are universally invalid—Node.js still uses callbacks in many APIs—but that promise-based Mongoose operations are conveniently handled with async/await.

10. Testing the endpoint in Postman

The lecture uses Postman to send a request to the local Express server. To test a create endpoint:

• Start the MongoDB service/server.

• Start the Node.js application.

• Choose the POST method and enter the local URL, such as http://localhost:3000/person.

• Open Body, select raw, and choose JSON.

• Send a JSON object with fields that match the schema.

• Inspect the response status and response body.

Example request body:

```
{
  "name": "Aarav",
  "age": 25,
  "role": "chef",
  "mobile": "9876543210",
  "email": "aarav@example.com",
  "address": "Delhi",
  "salary": 35000
}
```

If the request succeeds, the response should contain the saved document (including MongoDB's generated identifier). Verify the record in MongoDB Compass. If it fails, read the server terminal error as well as the Postman response.

11. HTTP status codes mentioned

• **200 OK:** the request succeeded. The lecture uses it for a successful save response.

• **500 Internal Server Error:** the server encountered an unexpected problem while processing the request.

For a REST API, 201 Created is often a better choice for a successful POST that creates a new resource. Other useful codes include 400 Bad Request for invalid input, 404 Not Found for a missing resource, and 409 Conflict for a uniqueness conflict. These extra examples are general API guidance rather than the core code shown in the transcript.

12. MenuItem model and restaurant data

Toward the end, the lecture applies the same schema/model idea to menu data. A MenuItem model represents one menu item stored in the restaurant application's database. The transcript indicates that a menu schema is provided as homework/reference, and that students should create endpoints such as POST /menu and GET /menu to interact with it.

The exact complete field list for MenuItem is not fully recoverable from the transcript excerpt, so do not assume a specific schema beyond the fields in your class-provided file. Follow the schema shared by your instructor.

The important reusable pattern is the same:

```
const menuItem = new MenuItem(req.body);
const savedMenuItem = await menuItem.save();
```

For a real application, validate the submitted fields, handle duplicate or malformed values, and return appropriate status codes. A GET endpoint should retrieve menu items from MongoDB rather than merely echoing a hard-coded response.

13. CRUD: what the lecture expects next

CRUD means Create, Read, Update, and Delete. The lecture focuses on connecting the database and implementing the create/save flow, then explains that the same model can support the remaining operations.

• **Create:** accept data and save a new document, usually through POST.

• **Read:** retrieve documents, commonly through GET.

• **Update:** change selected fields of an existing document, commonly through PATCH or PUT.

• **Delete:** remove a document, commonly through DELETE.

The transcript frames GET, update, and delete routes as the next extension/homework. Exact route names and implementation details should follow the class assignment.

14. Common errors and debugging checklist

• **Cannot connect to MongoDB:** verify the database service is running, the URI/port is correct, and the database server is reachable.

• **req.body is undefined:** register express.json() before routes and select JSON in Postman.

• **Route returns 404:** confirm the HTTP method and path match the registered route.

• **Validation error:** check required fields, field types, and allowed enum values.

• **Duplicate key error:** check whether a field with a unique index already exists.

• **Server crashes or response hangs:** await the database operation and handle rejected promises in try/catch.

• **Record not visible in Compass:** connect Compass to the same MongoDB server and inspect the correct database and collection.

The transcript demonstrates stopping and restarting the MongoDB service to show connection events. If the database service stops while the app is running, the connection may be lost; restarting the service can allow reconnection, but always verify the current state and inspect logs.

15. Recommended project checklist

• Install Mongoose and confirm it appears in package.json.

• Create db.js with the MongoDB URI, connection setup, event listeners, and export.

• Import the database module near the beginning of server startup.

• Enable JSON body parsing before declaring routes.

• Create the Person schema and model in a dedicated model file.

• Import the model in server.js.

• Create a POST route that reads req.body and awaits save().

• Wrap asynchronous database operations in try/catch.

• Test with Postman using raw JSON.

• Verify the saved document in MongoDB Compass.

• Create the MenuItem model based on the class-provided schema.

• Implement the assigned GET/POST routes and later extend to update/delete.

16. Quick revision questions

• Why is database connection logic placed in a separate db.js file?

• What role does Mongoose play between Node.js and MongoDB?

• What is the difference between a schema and a model?

• What do required, enum, and unique mean in a Mongoose schema?

• Why should JSON parsing middleware be registered before routes?

• What is req.body and where does its data come from?

• What does new Person(req.body) do?

• Why do we await save()?

• What does try/catch do when a database operation fails?

• How can you confirm that a POST request actually saved a document?

• What are the four CRUD operations, and which HTTP methods are commonly associated with them?

17. Final recap

The main learning outcome is the complete path from an incoming HTTP request to a MongoDB document: configure the Mongoose connection, define a schema, create and export a model, parse the request body, create a model instance, save it with async/await, and send a JSON response. Keep the connection, models, and route handlers organized in separate files. Then use Postman to test the API and MongoDB Compass to verify persisted data.